Lab 2: Introduction to hardware description language

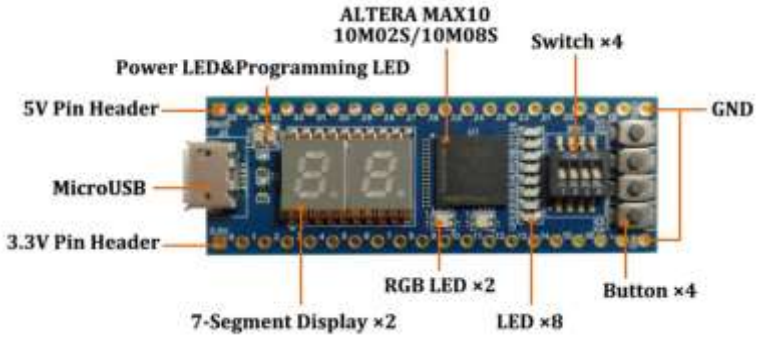
Hardware requried

- STEP-MAX10 development board presents a robust, portable and easy-to-learn hardware design platform built around the Altera MAX10 FPGA.

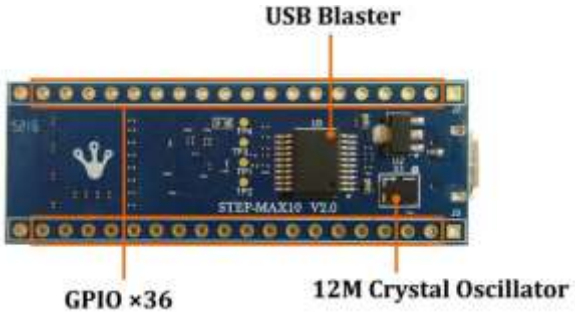
- MAX10 FPGA provide power consumption / cost and higher performance.



Block diagram of board

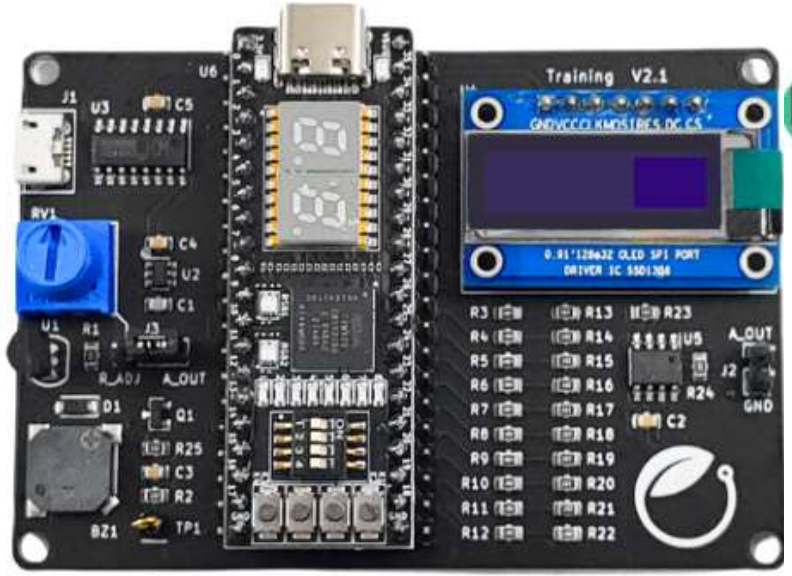


Development board (top view)

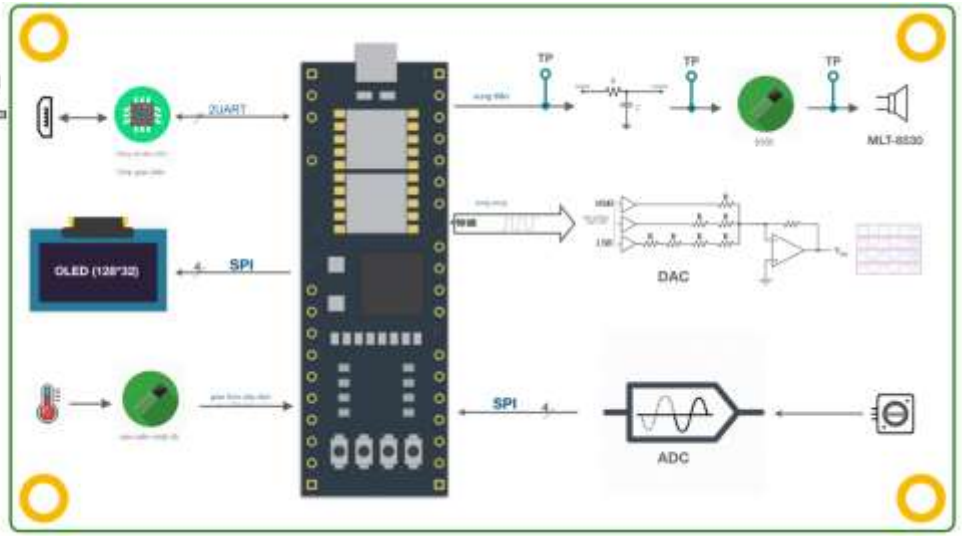


Development board (bottom view)

STEP MAX10 expansion board



STEP MAX10 expansion board



Block diagram of STEP MAX10 expansion board

Hardware description language

Hardware Description Languages (HDL) like SystemVerilog are used to describe hardware behavior
-> It can be converted to digital blocks made up of combinational and sequential elements.

SystemVerilog is an extension of Verilog with more features

Module

Definition:
Module is block of SystemVerilog code that implements a certain functionality
A module can be embedded within other modules and these modules can communicate with each other using their input + output ports.

Syntax:
module Module_name(input [data_type] port1, input [data_type] port2, output [data_type] port3);
//internal logic
endmodule

Declaration: Using *module* keyword, followed by module name, an optional port list; end of a module by *endmodule* keyword

Portlist: Specify input, output ports to module and their data types; portlist is optional.

Internal logic: Specify how module 's outputs are derived from its inputs: using operators, assignments, variables, statements, call other modules,.....

Data types and values of a variable

Basic data types available in SystemVerilog are shown in table below:

Nearly all SystemVerilog signals can be *logic,* except that signals with multiple drivers

Values SystemVerilog variables can hold

0	represents a logic zero, or a false condition
1	represents a logic one, or a true condition
x or X	represents an unknown logic value (can be zero or one)
z	represents a high-impedance state

	Data type	2-state/4-state	# Bits	signed/unsigned	C equivalent	
SYSTEM VERILOG	reg	4	>= 1	unsigned		VERILOG
	wire	4	>= 1	unsigned		
	integer	4	32	signed		
	real				double	
	time					
	realtime				double	
VERILOG	logic	4	>= 1	unsigned		
	bit	2	>= 1	unsigned		
	byte	2	8	signed	char	
	shortint	2	16	signed	short int	
	int	2	32	signed	int	
	longint	2	64	signed	long int	
	shortreal				float	

Operator and precedence

SystemVerilog operators and precedence of operators are shown in below table:

Some types of operators:

- Arithmetic operators
- Relational operators
- Bitwise operators
- Conditional operator
- Assignment operators
  + Blocking assignment
  + Non-blocking assignment

	Op	Meaning
Highest	~	NOT
	*, /, %	MUL, DIV, MOD
	+, -	PLUS, MINUS
	<<, >>	Logical Left/Right Shift
	<<<, >>>	Arithmetic Left/Right Shift
	<, <=, >, >=	Relative Comparison
	==, !=	Equality Comparison
	&, ~&	AND, NAND
	^, ~^	XOR, XNOR
	|, ~|	OR, NOR
Lowest	?:	Conditional

Continuous assignment

Continuous assignment: value of the left-hand side signal is continuously updated to match the value of the right-hand side expression
assign statement is used to create continuous assignment in hardware description.

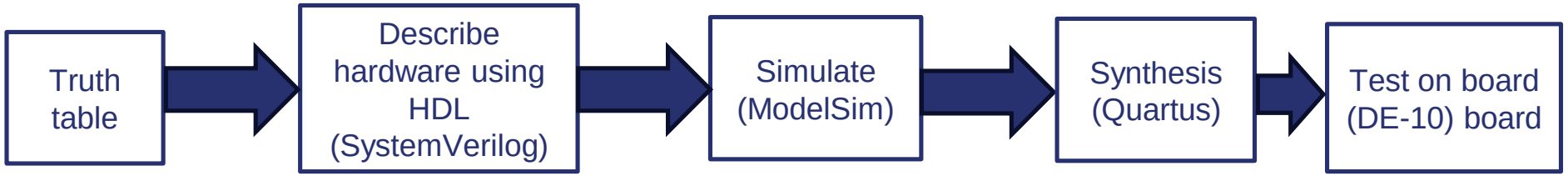
Syntax:

```
assign LHS = RHS
```

LHS: signal need to assign a value to
RHS: signal or expression

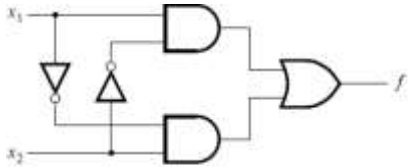
Design flow

In this course: To design a digital circuit



Truth table and SystemVerilog

Truth table



x_1	x_2	f
0	0	0
0	1	1
1	0	1
1	1	0

Figure 12. The light controller circuit.

System Vertilog

```
module Light (input logic x1, x2, output logic f);

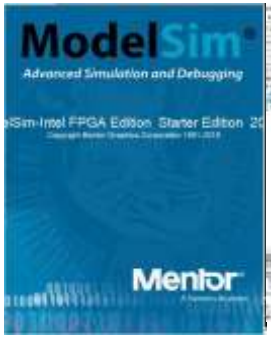
    assign f = ~((x1 & ~x2) | (~x1 & x2));

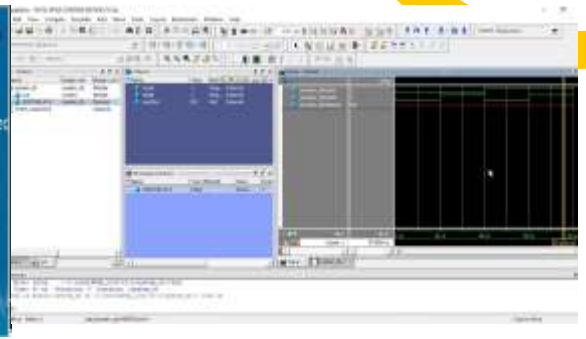
endmodule
```



Simulation and Synthesis

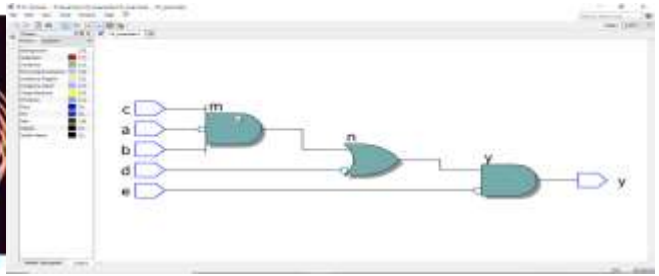
Simulation
To verify the module operates correctly or not
By applying inputs to a module and checking outputs





Synthesis
Textual description of a module is transformed into logic gates.





Writing testbench

Testbench: a SystemVerilog module to verify the functionality of a design through simulations.

Steps:

- Top level hardware design module is instantiated within testbench
- Declare testbench component signals and connect them to design ‘s input/output ports
- Inputs to the design are driven with certain values
- Outputs of design are analyzed and compared with expected values to check if the design behavior is correct or not

Writing testbench (cont)

Structure of a testbench:

```
// All verification components are placed in this top testbench module
 Module tb_top();
// Declare testbench component signals that need to be connected to the design instance
 // These signals are assigned some values that in turn gets transferred to
// the design as inputs because they are connected with the ports in the design
logic a, b, y;
// Instantiate the design module and connect the testbench component signals declared
above with the ports in the design

mydesign DUT(a,b,y);
//Assign values to testbench component signals  that can be diven to the design
```

DUT stans for design under test and is top hardware design module written in SystemVerilog

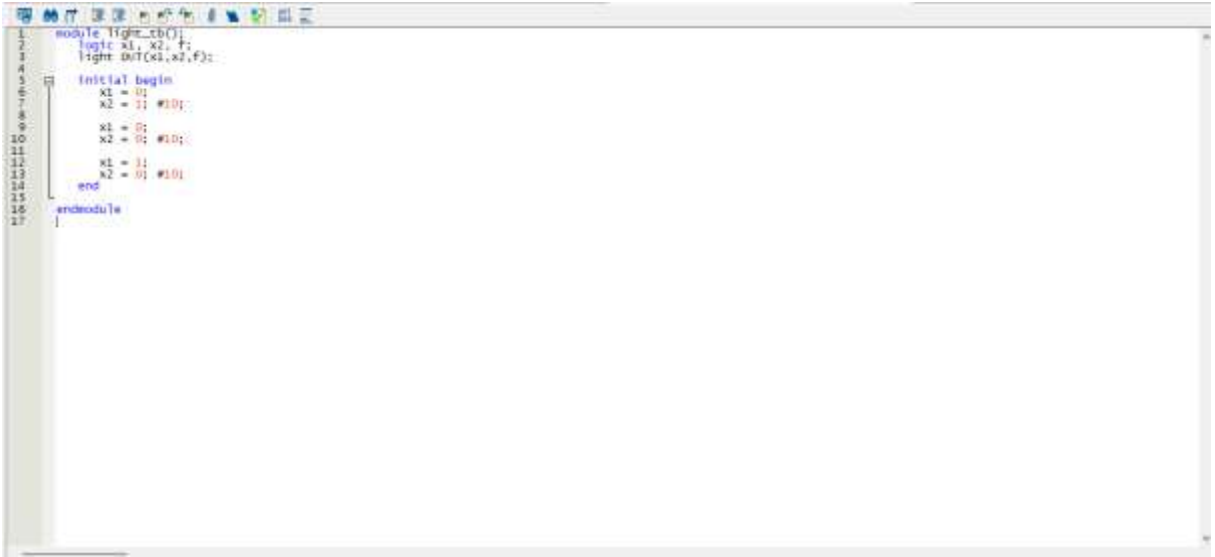
Example: inverter (cont)

An example testbench for inverter design is given below:

```
module light_tb();
    logic x1, x2, f;
    light DUT(x1,x2,f);
    initial begin
          x1 = 0;
          x2 = 1; #10;

          x1 = 0;
          x2 = 0; #10;

          x1 = 1;
          x2 = 0; #10;
    end
Endmodule
```

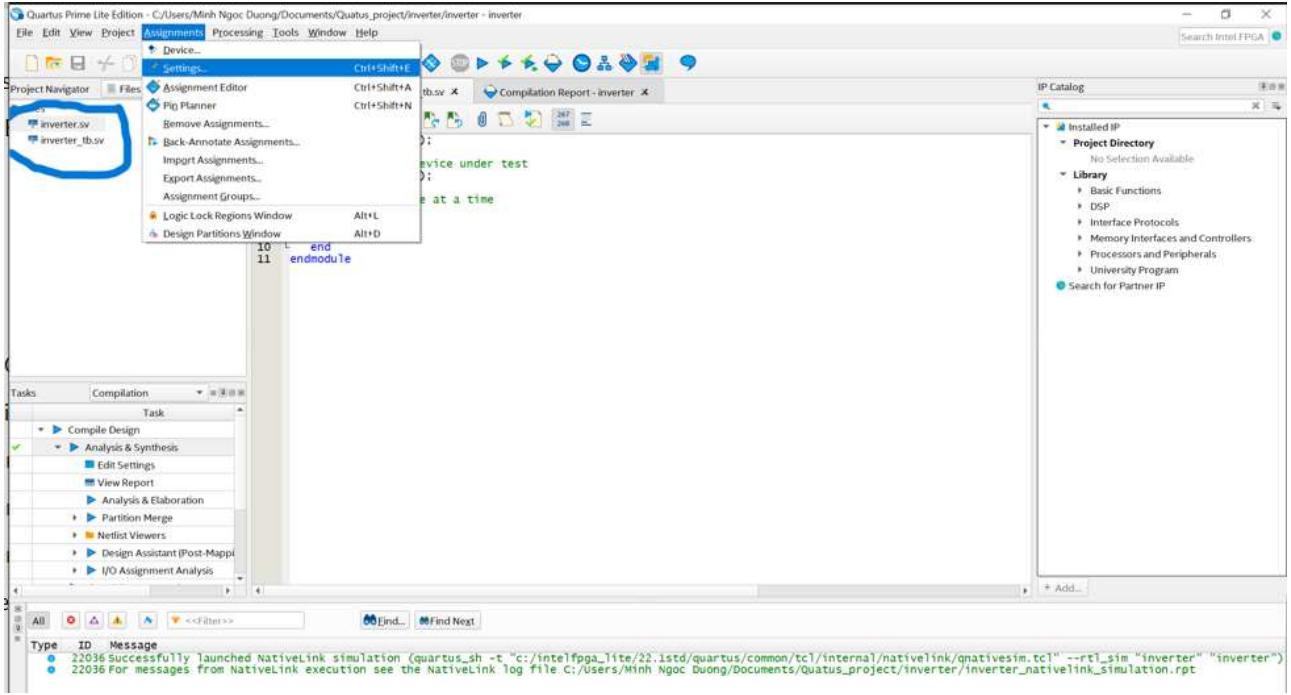


delay:#10 – delay 10 time units

initial: define a block of code that is execute once at the beginning of a simulation; used for initialization, testbench setup,...

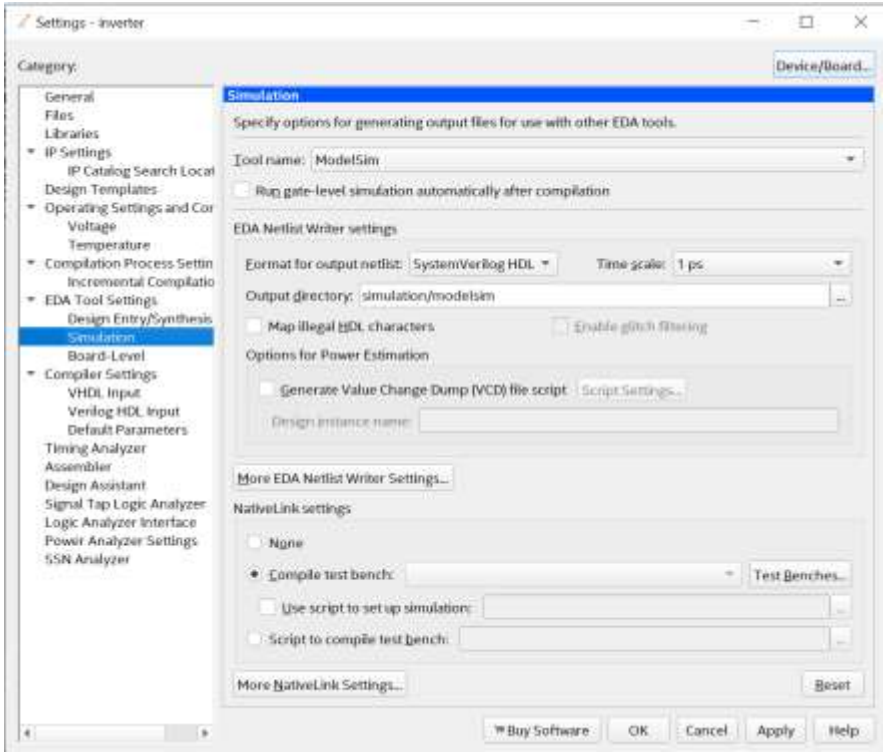
Run simulation in Quaturs

After add inverter.sv and inverter_tb.sv file to current project -> run simulation
Choose Asignments -> Settings to add testbench file



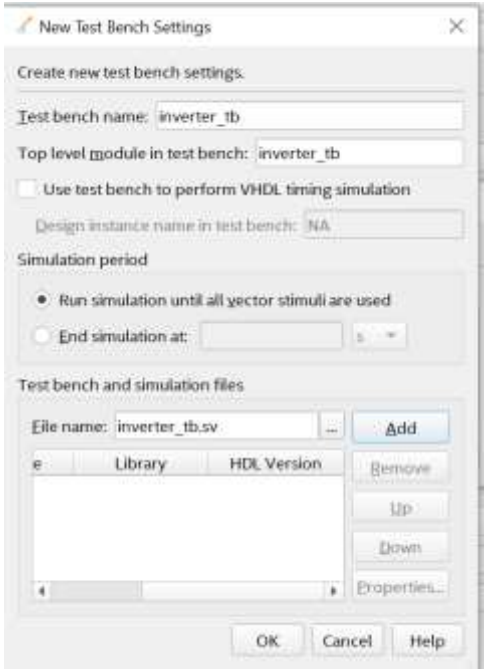
Run simulation in Quaturs (cont)

In Settings, go to Simulation and configure as picture below



Run simulation in Quaturs (cont)

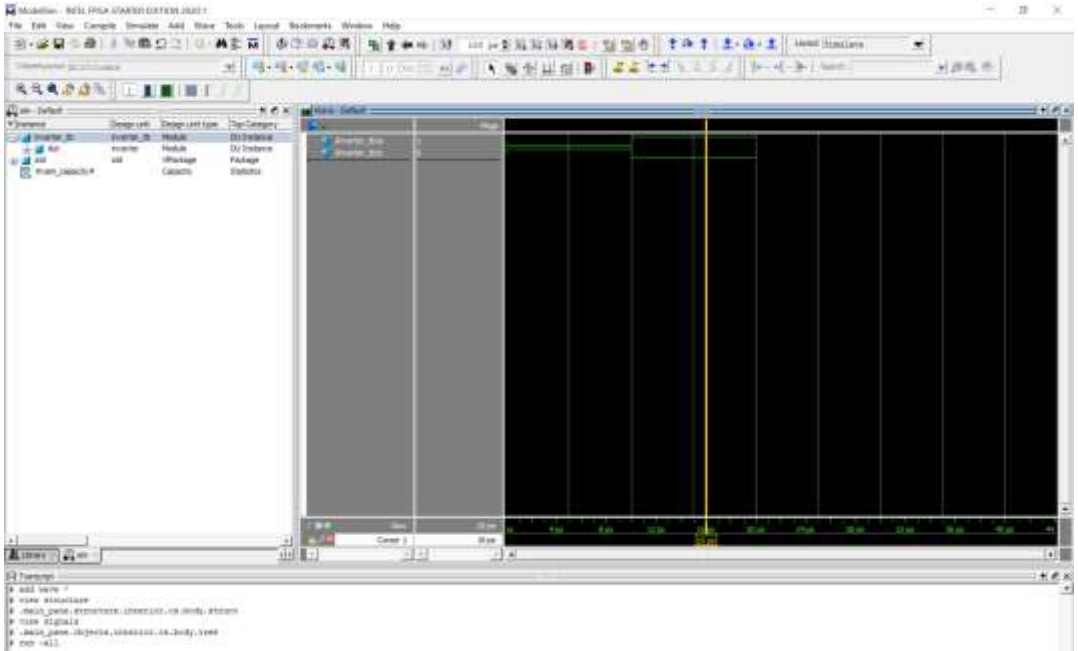
Click on TestBenches -> New
Enter test bench name, top level module in test bench
In Filename, browse to *inverter_tb.sv* file in your computer and click Add -> OK



Run simulation in Quaturs (cont)

Add path to ModelSim: go to Tools -> Options -> EDA Tool Options -> add path to ModelSim
Path has format:
C:/intelFPGA_lite/20.1std/modelsim_ase/win32aloem

Choose Tools -> Run Simulation Tool -> RTL Simulation to lauch ModelSim and observe result of simulation

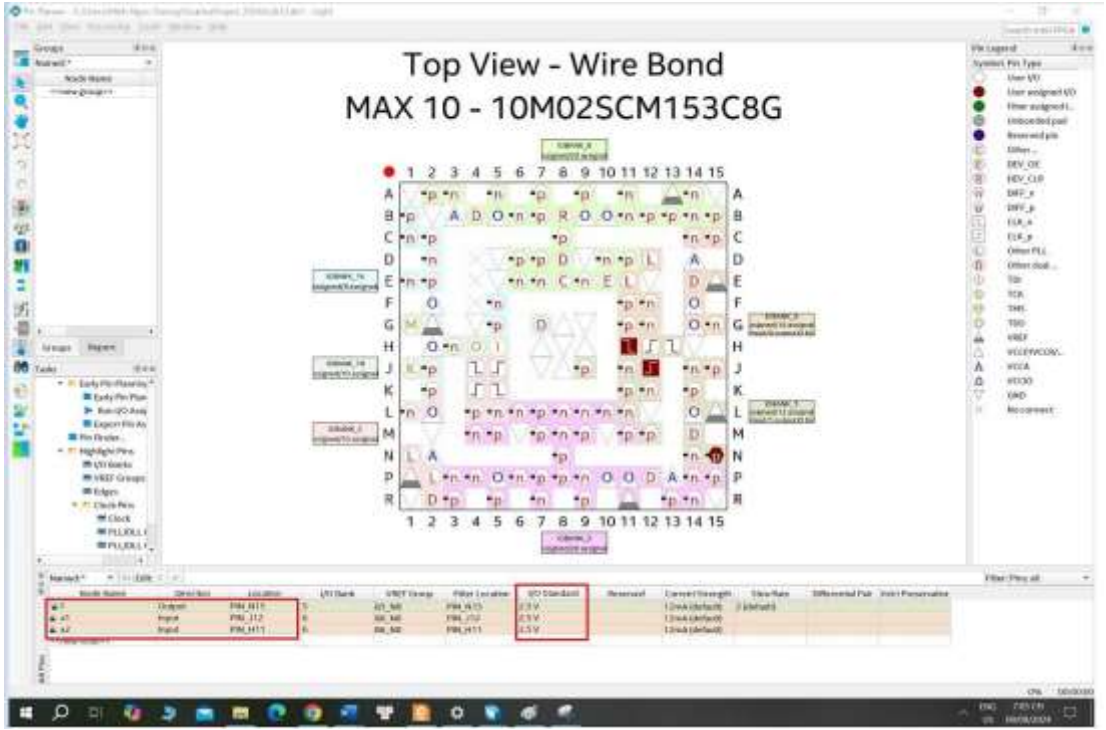


Pin assignments

STEP PINs	FPGA PINs	STEP PINs	FPGA PINs	Digital Display1	FPGA PINs	12M CLOCK	FPGA PINs
3.3V		VBUS		SEG-A1	E1	PCLK	J5
GPIO0	M4	GPIO35	B4	SEG-B1	D2	LED	FPGA PINs
GPIO1	P3	GPIO34	A5	SEG-C1	K2	LED1	N15
GPIO2	M5	GPIO33	A7	SEG-D1	J2	LED2	N14
GPIO3	R3	GPIO32	B6	SEG-E1	G2	LED3	M14
GPIO4	L6	GPIO31	E7	SEG-F1	F5	LED4	M12
GPIO5	P4	GPIO30	D7	SEG-G1	G5	LED5	L15
GPIO6	L7	GPIO29	B7	SEG-DP1	L1	LED6	K12
GPIO7	R5	GPIO28	C8	SEG-DIG1	E2	LED7	L11
GPIO8	P6	GPIO27	B8	Digital Display2	FPGA PINs	LED8	K11
GPIO9	R7	GPIO26	D10			Switch	FPGA PINs
GPIO10	P7	GPIO25	A9	SEG-A2	A3	SW1	J12
GPIO11	P8	GPIO24	A11	SEG-B2	A2	SW2	H11
GPIO12	P9	GPIO23	A13	SEG-C2	P2	SW3	H12
GPIO13	R9	GPIO22	B11	SEG-D2	P1	SW4	H13
GPIO14	R11	GPIO21	A14	SEG-E2	N1	Button	FPGA PINs
GPIO15	P12	GPIO20	B13	SEG-F2	C1	KEY1	J9
GPIO16	R14	GPIO19	B14	SEG-G2	C2	KEY2	K14
GPIO17	P15	GPIO18	B15	SEG-DP2	R2	KEY3	J11
GND		GND		SEG-DIG2	B1	KEY4	J14
RGB LED1	R	G	B	RGB_LED2	R	G	B
FPGA PINs	G15	E15	E14	FPGA PINs	C15	C14	D12

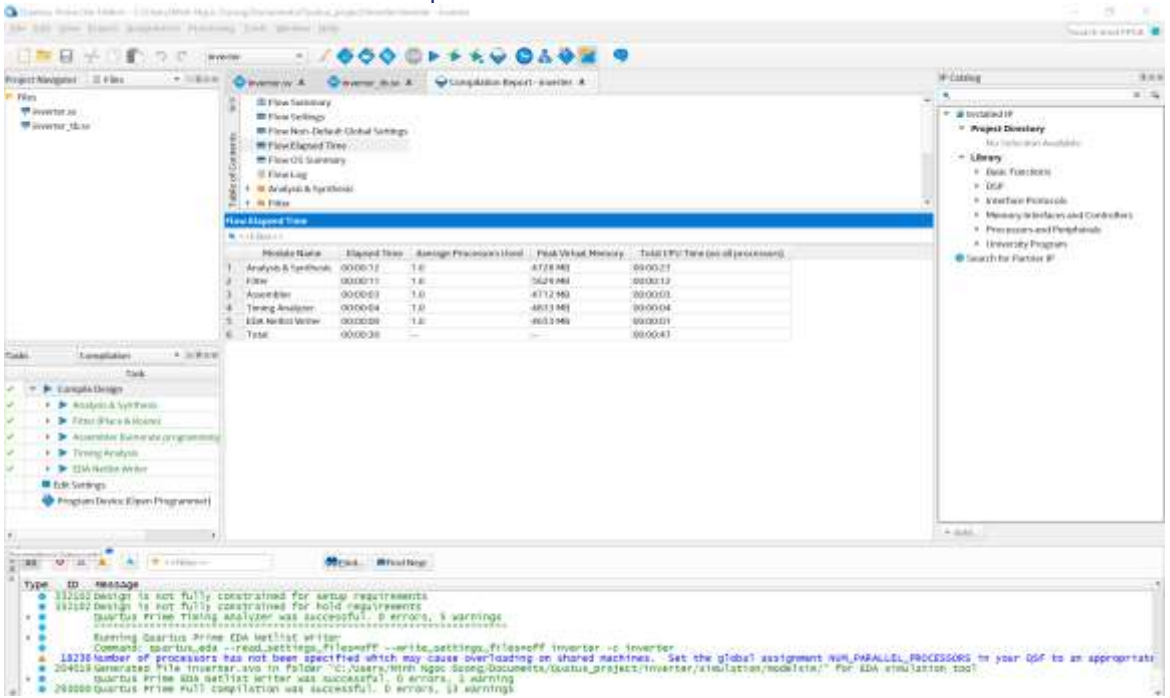
Test design module in DE-10 board (cont)

Go to Assignments -> Pin Planner -> assign pins of design module on DE-10 board



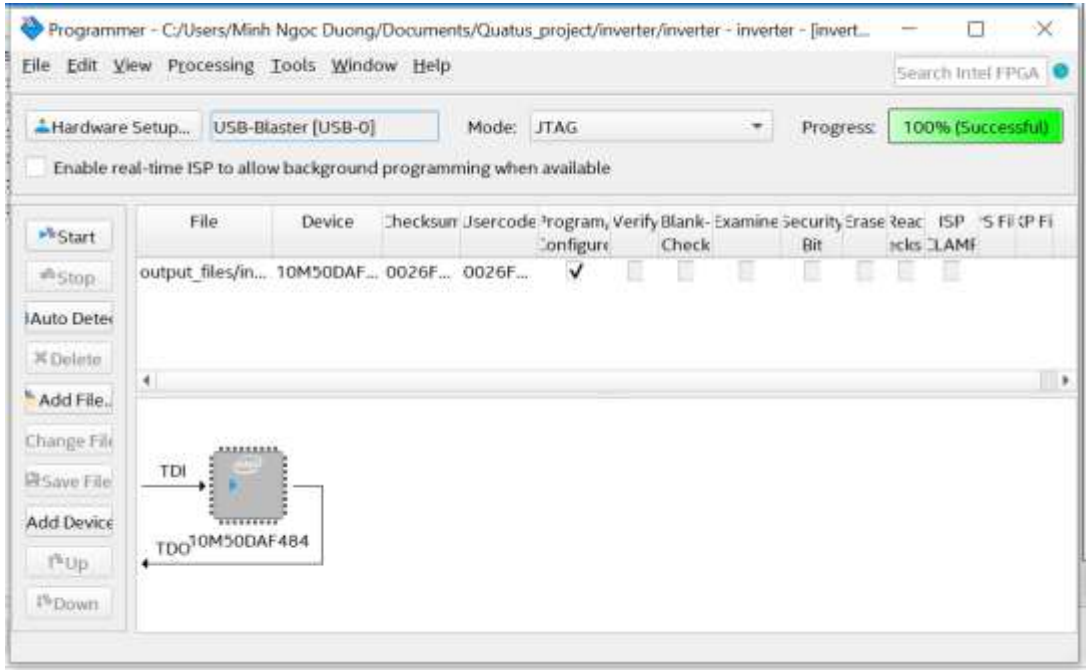
Test design module in DE-10 board (cont)

Do complete Compline step in design
Right click on Compile Design -> Start
All green check mark -> done Compline



Test design module in DE-10 board (cont)

Go to Tool -> Programmer
After select Hardware , click on Start
Progress bar is full with green -> Done
Check operations of design on DE-10 board



Write testbench using testvector

Create test vector file:

File -> New -> Text file, save file as *testVectors.txt*, enter testvector as below:

input1_input2_....._expectedOutput1 (Case 1)

input1_input2....._expectedOutput1 (Case 2)

Example with inverter:

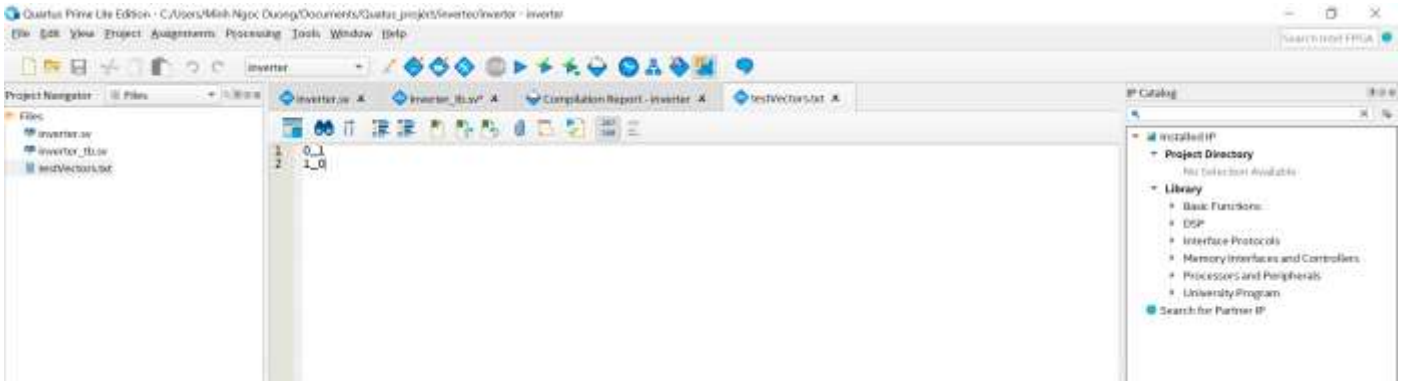
0_1

1_0

Explain:

Line1: 0 is value of x, 1 is value of expected y (expected output) (Case 1)

Line2: 1 is value of x, 0 is value of expected y (expected output) (Case 2)



Exercises

Ex1: Design a NAND gate using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.

Ex2: Design a 2:1 multiplexer using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.

Ex3: Design a 4:1 multiplexer using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.